OBJECTIU

• Transformacions per a la correcció del punt de vista
• Enfocaments d'aprenentatge automàtic (ML) per a la segmentació d'imatges: mètodes de llindar (*thresholding*), *watershed*
• Detectors de *blobs* com a detectors d'objectes: escala, anisotropia, orientació
• Implementar un mètode per a la segmentació de caràcters de la matrícula basat en *blobs* i *watershed*
• Implementar una funció que retalli els caràcters de la matrícula a partir de la seva segmentació

Exercici 1. Segmentació bàsica de caràcters mitjançant llindars. Carregueu les matrícules retallades i el fitxer PlateRegions.npz:

```
data=np.load('PlateRegions.npz',allow_pickle=True)
#utilitzeu data.files per obtenir els noms de les variables de dades
regionsImCropped=data['regionsImCropped']
```

Executeu `detectCharacterCandidates` a `LicensePlateCharacterSegmentation.py` utilitzant les caixes amb coordenades respecte a la matrícula retallada (`regionsImCropped`).

a) Identifiqueu els passos principals (preprocessament, processament/segmentació i filtratge de postprocessament) del mètode `detectCharacterCandidates`, visualitzeu els resultats intermedis i la sortida.

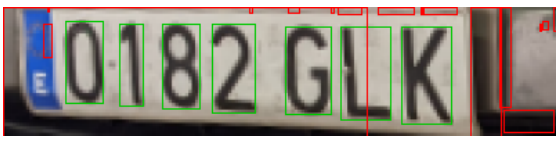
1. Preprocessament
   1.1. **transformació de perspectiva** (four_point_transform) corregeix la inclinació.
   1.2. **resize** per redimensionar l'amplada de 400 píxels.
   1.3. passa a HSV i obté el canal V (**brillantor**) per separar els caràcters del fons.



2. Processament i segmentació
   2.1. **llindar adaptatiu**: compara cada píxel amb la mitjana del seu voltant. Els caràcters queden blancs, encara que la il·luminació no sigui uniforme.
   2.2. **operacions morfològiques**: s'utilitzen un tancament (closing) i una obertura (opening). Es fan dos iteracions de cada amb un kernel vertical 1x3. El tancament tapa forats dins dels caràcters, i l'obertura treu soroll prim.



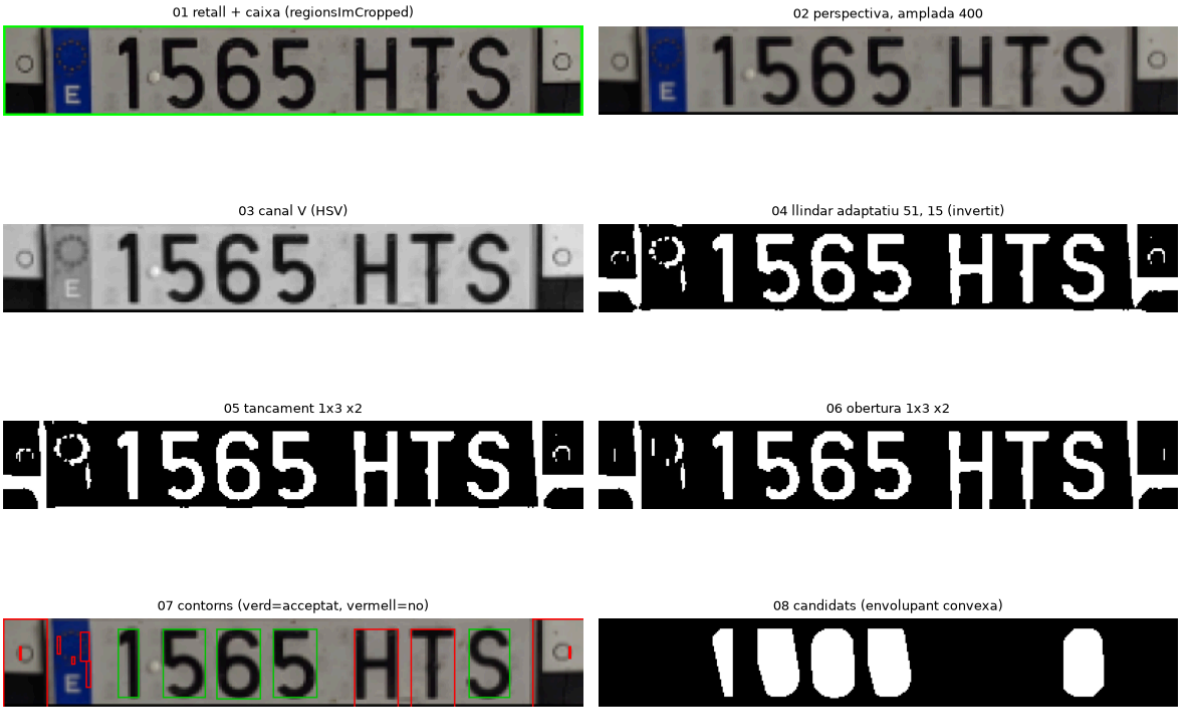
3. Postprocessament
   3.1. **cv2.findContours** troba els contorns de les regions blanques.
   3.2. **filtre**: es queda amb les taques que no toquen les vores de la matrícula i es conserven les que compleixen les condicions:
      - L'àrea del contorn és superior a 300 píxels.
      - L'alçada del contorn està entre el 50% i el 90% de la de la matrícula.



3.3. es calcula l'embolcall convex (**convex hull**) de cada contorn acceptat, que genera una màscara per a cada potencial caràcter.



Per un retall molt ajustat als caràcters hi ha perill que el tancament i obertura provoqui que els contorns toquin la vora i, llavors les taques no acabin passant el filtre, com passa per aquest retall:



A les 69 matrícules retallades (els 98 retalls, sense les còpies de la mateixa foto) surten exactament 7 candidats en 18 de 19 frontals i en 50 de 50 laterals. L'única que falla és 1565HTS, el retall ajustat d'abans (61 px d'alçada), que dona 5 candidats.

b) Quines parts de `detectCharacterCandidates` es podrien substituir per un detector de *blobs*?

Un detector de blobs (per exemple, un LoG a l'escala d'un caràcter) busca directament regions fosques de la mida d'un caràcter. Per tant, pot substituir:

- El llindar adaptatiu i la morfologia: el LoG ja respon a taques fosques sobre fons clar, sense haver de binaritzar.
- La cerca de contorns i el filtre d'àrea i d'alçada: l'escala σ del blob fixa la mida. Les línies del marc, el soroll petit i les zones molt grans no donen màxim a l'escala d'un caràcter.

El preprocés (perspectiva i canal V) es manté perquè el detector també necessita la matrícula redreçada i a una mida coneguda per triar σ. També cal un pas final per passar del

blob (centre i radi) a la regió del caràcter. Pot ser una caixa al voltant del centre, o el watershed de l'Ex3, que fa servir els blobs com a llavors.

Exercici 2. Detectors de *blobs*. Detecteu les regions que contenen cada caràcter utilitzant detectors de *blobs*:

a) Apliqueu l'operador LoG de la biblioteca de Python `skimage` (https://scikit-image.org/docs/0.25.x/auto_examples/features_detection/plot_blob.html) a les matrícules retallades. Quins paràmetres de la gaussiana es poden ajustar? Què retorna? Visualitzeu els *blobs* detectats en una matrícula en vista frontal i lateral. S'ajusten als caràcters? Discutiu les possibles fonts d'error i com es podrien resoldre.

Primer de tot hem buscat què és. El filtre LoG és una "lupa de taques" que il·lumina els llocs de la imatge on hi ha una taca fosca d'una mida concreta. La mida la fixa σ, i n'hi ha dues: una per a l'alçada i una per a l'amplada.

Els paràmetres són:

paràmetre	per defecte	Què controla
min_sigma, max_sigma	1, 50	Rang d'escales de la gaussiana. Poden ser un sol valor o per un eix (gaussiana anisòtropa).
num_sigma	10	Quantes escales es proven entre el mínim i el màxim.
log_scale	False	Escales repartides en escala logarítmica en lloc de lineal.
threshold / threshold_rel	0,2 / -	Resposta mínima (abs o relativa al màx) perquè un punt sigui un blob.
overlap	0,5	Si dos blobs se solapen més d'aquesta fracció, s'elimina el més petit.
exclude_border	False	No accepta blobs a prop de la vora.

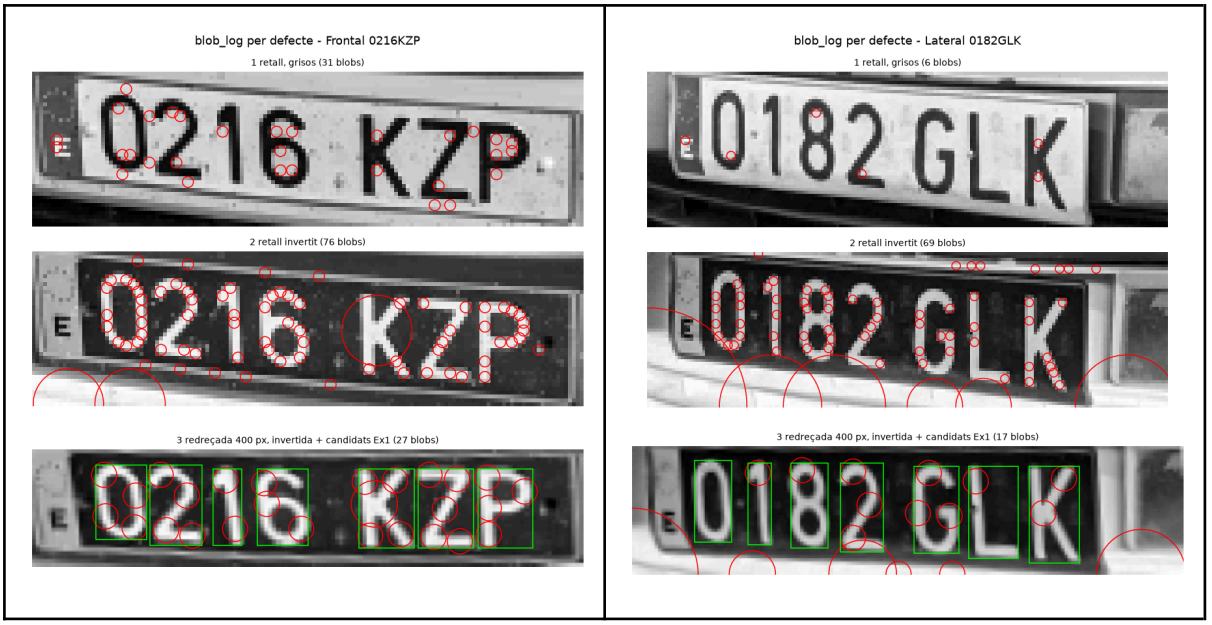
Retorna una matriu amb una fila per blob: (fila, columna, σ), és a dir, el centre i l'escala on la resposta és màxima. El radi del blob és aproximadament $\sigma \cdot \sqrt{2}$.

Hem passat el detector LoG sobre les matrícules retallades. Les hem preprocessat de 3 maneres diferents:

- retall original a gris,
- retall a gris invertit (ja que busca taques clares sobre fons fosc)
- i el retall invertit i redreçat com feia detectCharacterCandidates (perspectiva + amplada).

Visualitzem els blobs amb els paràmetres per defecte:

matrícula Frontal	matrícula Lateral



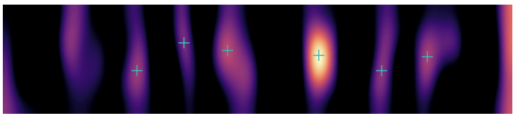
Amb els paràmetres originals els blobs no encaixen gaire. El panell 3 és el millor cas, on hi estem visualitzant en verd la caixa treta de l'exercici anterior.

Les possibles fonts d'error i les seves solucions serien:

- Que blob_log busca taques clares sobre fons fosc i a les matrícules els caràcters són foscos. A la visualització la imatge sense invertir té els blobs entre els caracters als forats clars. La solució és **invertir** la imatge.
- Amb la resolució original els traços fan pocs píxels i només surten blobs petits que són soroll. La solució és **ampliar** i redreçar la matrícula.
- Amb el rang 1-50 i 10 valors les proves estan molt separades. La majoría no té res a veure amb la mida d'un caràcter. La solució és fixar min_sigma i max_sigma a partir de la mida dels caràcters, que es pot treure de les caixes de l'Ex1.
- La gaussiana és circular i un caràcter és més alt que ample. Un cercle no s'hi ajusta, i el detector troba les parts arrodonides del traç en lloc del caràcter sencer. La solució és una gaussiana anisòtropa amb una σ vertical més gran que l'horitzontal (σ per eix a blob_log, o gaussian_laplace de scipy, Ex2b)
- Les zones fosques del cotxe (marc, para-xocs) es tornen clares en invertir i donen blobs grossos. La solució seria retallar la matrícula ajustada, filtrar els blobs per posició i mida i fer un preprocés morfològic que només realci traços foscos i prims, com el blackhat.
- El llindar absolut depèn del contrast de cada foto. La solució seria fer servir un threshold relatiu o normalitzar la imatge.

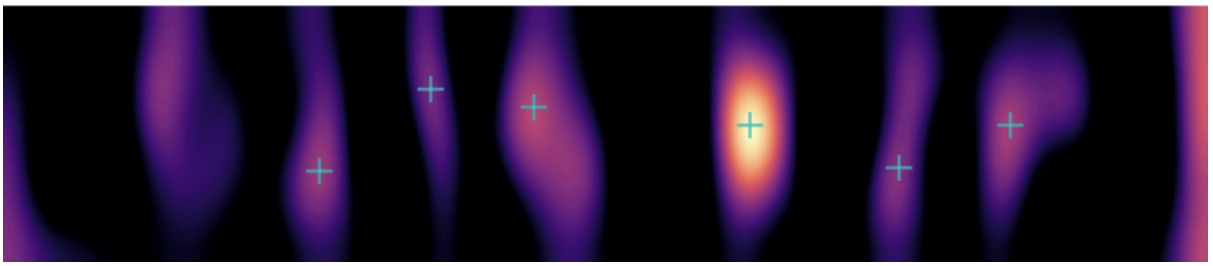
b) Repetiu el punt 2.a) utilitzant la funció `gaussian_laplace` de la biblioteca de Python `scipy.ndimage` (https://docs.scipy.org/doc/scipy/reference/ndimage.html). Quins són els seus paràmetres d'entrada i sortida en comparació amb l'operador de `skimage`? Apliqueu-la a les matrícules retallades. Ajusteu l'escala sigma a la mida dels caràcters (pista: utilitzeu les propietats de les segmentacions de l'Exercici 1). Feu un postprocessament de la imatge de sortida per obtenir un conjunt de regions candidates a caràcter.

Comparació amb skimage:

	skimage.feature.blob_log	scipy.ndimage.gaussian_laplace
Què és	Un detector de blobs complet	Només un filtre
Escales	En prova de diverses	Una sola σ, que pot ser diferent per eix
Altres params	threshold, overlap, exclude_border	mode i cval (què fer a la vora), truncate (mida del filtre)
Sortida	Llista de blobs (fila, col, σ)	Una imatge de la mateixa mida amb la resposta del filtre: 

Per triar la σ hem fet servir les caixes dels caràcters de l'Ex1, només a les matrícules de train on va trobar 7 caixes (329 caixes). Com que la matrícula està redreçada a 400 px d'amplada, la mida mediana d'un caràcter és de 34 × 55 px. Una taca de radi r respon més fort a σ = r/√2, i d'aquí surten σx = 12,0 i σy = 19,4.

Com ja hem mencionat el resultat del filtre és una imatge de la mateixa mida amb una franja brillant per caràcter (les creus marquen el punt més brillant de la franja):



Com que scipy retorna un filtre però no les caixes, hem de fer nosaltres el postprocessament. Hem provat dues maneres de passar aquesta sortida a caixes:

1. Llindar d'Otsu i regions que no toquen la vora. Les franges són tan altes com la matrícula i toquen la vora, per això el filtre de detectCharacterCandidates elimina la majoría dels blobs.



(en aquest cas queden 4 regions i se n'eliminen 5 per tocar la vora)

2. Cada màxim local es pren com el centre d'un caràcter, i agafem la caixa de 34 x 59 px al voltant. La distància mínima entre màxims és de mig caràcter, i no s'accepten màxims a menys de mig caràcter de la vora.



Els resultats sobre les matrícules de train on l'Ex1 troba 7 caràcters són:

Postprocés	Caràcters trobats un sol cop	Caràcters trobats un sol cop	Matrícules amb 7 regions
Otsu + vora	25,7 %	38,8 %	0/15 frontals, 4/32 laterals
Màx locals	64,8 %	68,8 %	1/15 frontals, 6/32 laterals

El LoG troba de manera aproximada on és cada caràcter, però no en dona la caixa exacta.

- La resposta de cada caràcter és una franja allargada. El màxim pot caure a qualsevol alçada i la caixa queda desplaçada.
- Alguns caràcters no tenen màxim (el "0" de 0216KZP).
- Algunes respostes venen de fora de la matrícula (la vora dreta de 0182GLK).

c) Apliqueu un cert preprocessament morfològic per ressaltar els caràcters de la matrícula i compareu el rendiment del detector de taques (*blob detector*).

Hem aplicat un black-hat (tancament menys imatge) al canal V de la matrícula redreçada. El tancament esborra les estructures fosques més primes que l'element estructurant, que són els traços. En restar la imatge, queden els traços clars sobre fons negre, i les zones fosques grans desapareixen. L'element és un quadrat de 34 × 34 px, l'amplada mediana d'un caràcter a train, perquè els traços hi caben i les zones més grans que un caràcter no. La σ i el postprocés de màxims locals són els de l'Ex2b, i com que ara els caràcters són clars, fem servir −LoG.
Resultats (matrícules on l'Ex1 troba 7 caràcters; percentatge de caràcters trobats per exactament una regió):

	Frontal train (n = 15)	Lateral train (n = 32)	Frontal val (n = 2)	Lateral val (n = 9)
Sense black-hat	64,8%	68,8%	85,7%	79,4%
Amb black-hat	62,9%	71,9%	85,7%	77,8%
Regions fora dels caràcters, sense → amb (train)	3 → 4	10 → 5	-	-
Matrícules amb 7 regions, sense → amb (train)	1 → 1	6 → 9	-	-



A les laterals, el black-hat neteja el fons: hi ha la meitat de regions fora dels caràcters (de 10 a 5), i més matrícules amb exactament 7 regions (de 6 a 9). A les frontals, que ja tenen poc fons, gairebé no canvia res. El percentatge de caràcters trobats es mou ±3 punts: puja a la lateral de train i baixa una mica a la frontal de train i a la lateral de validation. Amb tan poques matrícules (la frontal de validation en té 2), aquestes diferències no són concloents.

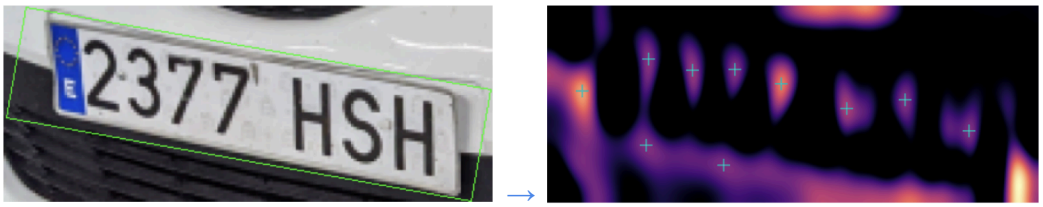
d) Creieu que el rendiment del filtre laplacià de gaussiana (*gaussian_laplace*) anisotròpic seria el mateix per a imatges preses des d'una vista lateral extrema? Quin banc de filtres gaussians seria necessari? Quin preprocessament podríeu aplicar perquè el filtre anisotròpic laplacià de gaussiana funcioni bé?

El filtre que hem fet servir és una el·lipse vertical de mida fixa (σx = 12,0, σy = 20,9) i només encaixa amb caràcters drets i tots de la mateixa mida. En una vista lateral extrema no passa:

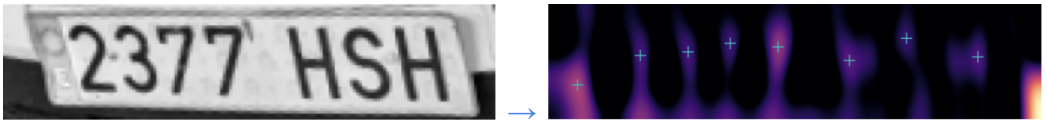
- Inclinació. La matrícula i els caràcters surten girats respecte dels eixos de la imatge, i el filtre no hi queda alineat.
- Perspectiva. Els caràcters del costat més proper a la càmera es veuen més grans que els del costat lluny, i una sola σ no s'ajusta a tots.
- Escurçament. Els caràcters es veuen més estrets, de manera que la σx triada és massa gran i la resposta de caràcters veïns es pot ajuntar.

Hem agafat la lateral de train més inclinada, 2377HSH (10° d'inclinació a la vora de dalt). Amb la mateixa σ i els mateixos màxims locals que a l'Ex2b:

- **sense redreçar**: surten 10 màxims. Quatre cauen sobre caràcters, però també n'hi ha a la franja blava i a la zona fosca del para-xocs de sota la matrícula.



- **redreçada**: surten 8 màxims, un per cada caràcter dels 7 més un a la franja blava de l'esquerra.



Quin banc de filtres caldria? Un conjunt de gaussianes anisòtropes que cobreixi:

1. diverses orientacions θ, per als caràcters girats;
2. diverses escales (σx, σy), perquè la mida dels caràcters canvia al llarg de la matrícula i perquè hi ha caràcters estrets com l'"1".

Per a cada píxel ens quedem la resposta més alta del banc. A més d'on és el caràcter, això diu quina orientació i quina mida té. gaussian_laplace només fa filtres alineats amb els eixos: per orientar-lo, cal girar la imatge (o construir el nucli a mà).

Un preprocessament que podria funcionar seria redreçar la matrícula amb una homografia a partir de les 4 cantonades reals (four_point_transform) i portar-la a una mida fixa. Així els caràcters queden drets i de la mateixa mida, i n'hi ha prou amb un sol filtre anisòtrop.

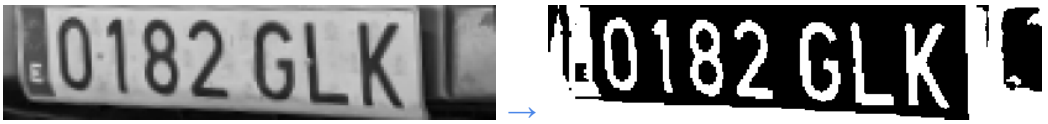
Aquí hi ha una limitació: les cantonades que dona el detector de detectPlates() són el rectangle girat de mínima àrea (minAreaRect). Totes les laterals de train tenen la mateixa alçada als dos costats (proporció 1,00). Amb aquestes cantonades, el redreçat només treu la rotació, però no la perspectiva, i els caràcters continuen una mica inclinats. Per corregir del tot la perspectiva calen les 4 cantonades reals de la matrícula. Les podria donar l'anotació o un detector que trobi els vèrtexs del quadrilàter.

Exercici 3. Segmentació per conques hidrogràfiques (*Watershed*).

a) Executeu `WaterShedExampleCode.py` sobre una imatge retallada de la matrícula, tant frontal com lateral. Visualitzeu les imatges intermèdies per identificar les operacions rellevants.

Hem executat WaterShedExampleCode.py sobre retalls redreçats a 400 px a grisos per a matrícules Frontals i Laterals. Els passos són:

1. Otsu invertit → els caràcters foscos passen a blanc.



2. Opening 3x3 (2 iteracions) → treu soroll i separa caràcters que es toquen.



3. Distance Transform → Cada píxel blanc val la distància al negre més proper. El que supera el 50% del màxim és el primer pla segur, i la resta del blanc és la zona dubtosa.



(primer pla segur) (zona dubtosa)



4. Llavors → Una llavor per tros de primer pla segur, una per al fons, i zero a la zona dubtosa.



5. Watershed → Inunda la zona dubtosa des de les llavors i marca les fronteres.



Cap caràcter no queda segmentat.
Les operacions rellevants són:

- El llindar Otsu. També marca com a primer pla les zones fosques de fora de la matrícula: la franja i la vora a la frontal, el para-xocs i el cotxe a la lateral.

- El llindar del primer pla segur, relatiu al màxim de tota la imatge. Aquestes zones són molt més gruixudes que un traç i tenen la distància màxima. Només elles superen el 50 % del màxim, i els caràcters no tenen cap llavor.
- Les llavors. Sense llavor pròpia, els caràcters queden com a zona dubtosa, i el watershed els assigna al fons.

Per fer servir el watershed sobre matrícules caldria:
- treure les zones fosques grans abans del llindar;
- tenir una llavor per caràcter que no depengui del traç més gruixut de la imatge.

b) Apliqueu un cert preprocessament morfològic per ressaltar els caràcters de la matrícula abans d'aplicar el llindar (*thresholding*) a les imatges i compareu-ne el rendiment. És possible que hàgiu d'ometre alguns dels passos anteriors.

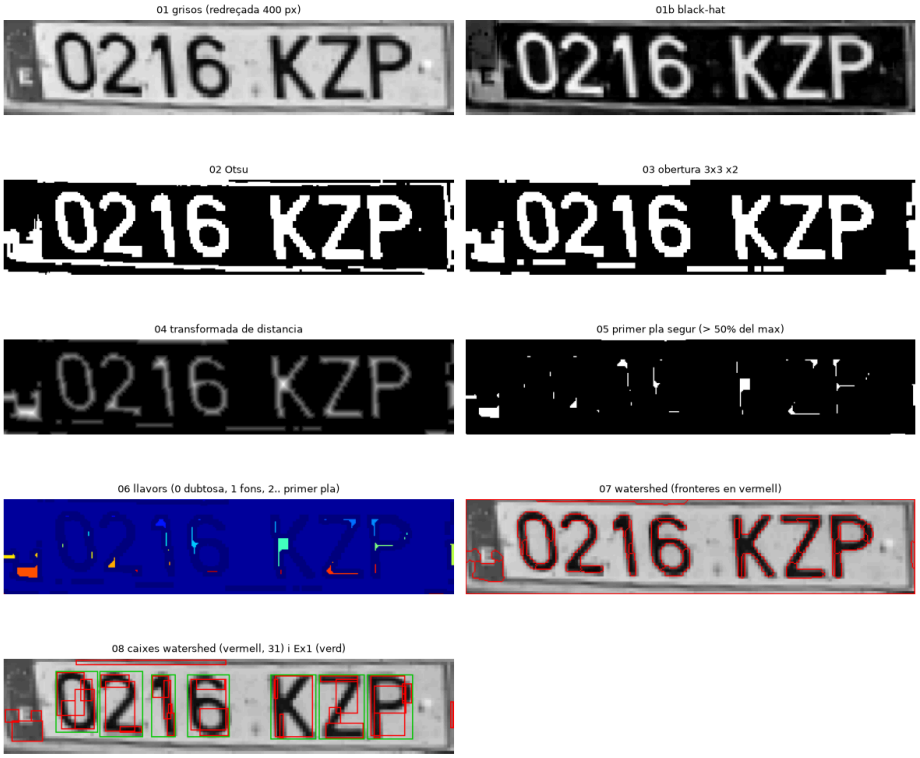
Com ja hem fet a l'Ex2c apliquem un black-hat que ja hem vist que elimina les zones fosques més grans que un caràcter, i és exactament on falla el watershed.
Ara a l'entrada els traços queden clars sobre negre, per tant fem l'Otsu sense invertir (el pas que ens saltem és la inversió). La resta del codi es manté igual: obertura, distància 50% del màxim, llavors i watershed.

Resultats (matrícules on l'Ex1 troba 7 caràcters; percentatge de caràcters amb 1 regió del watershed):

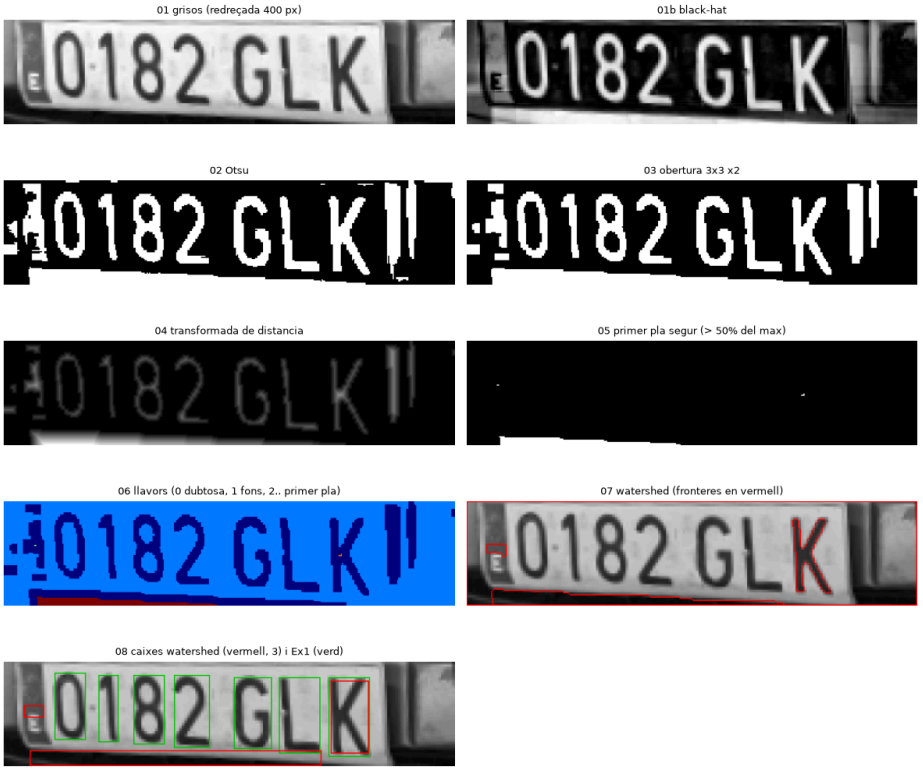
	Frontal train (n=15)	Lateral train (n=32)	Frontal val (n=2)	Lateral val (n=9)
Sense black-hat (Ex3a)	8,6%	9,8%	0,0%	9,5%
Amb black-hat	21,9%	8,5%	7,1%	15,9%
Regions fora dels caràcters, sense → amb (train)	33 → 41	44 → 100		

El black-hat elimina les zones fosques grans, i ara els caràcters entren al llindar. A les frontals de train es troben més caràcters (del 8,6 % al 21,9 %), però a les laterals de train gairebé no canvia (del 9,8 % al 8,5 %), i surten més del doble de regions que no són caràcters (de 44 a 100). Fora dels caràcters hi ha regions a totes dues vistes. El resultat continua sent dolent.

- Sobresegmentació (0216KZP, 33 llavors). Els traços tenen un gruix gairebé constant. El llindar deixa diversos trossets per caràcter, i cada caràcter es parteix en diverses regions.



- Infrasegmentació (0182GLK, 3 llavors). Una franja fosca sota la matrícula, més baixa que l'element del black-hat, no s'elimina. És més gruixuda que els traços, s'endú el màxim de distància, i els caràcters es queden sense llavor.



Per tant, el watershed necessita una llavor per caràcter que no depengui del gruix del traç.

c) Implementeu un mètode de postprocessament per obtenir una màscara binària dels caràcters.

Dels apartats anteriors en traiem dues conclusions:

- el LoG diu on és cada caràcter, però se'n deixa alguns (el "0", l'"1");
- el watershed dona la forma del caràcter, però no sap triar les llavors.

El mètode final fa servir com a llavors les taques de mida de caràcter, que sí que en donen una per caràcter, i després aplica el watershed.

1. Ressaltar els caràcters. Un black-hat de 34 × 34 px (l'amplada d'un caràcter) deixa només els traços foscos i prims, en clar. Les zones fosques grans desapareixen.



2. Blanc i negre. Llindar d'Otsu i, després, tancament i obertura amb un element vertical 1×3, com a l'Ex1. Cada caràcter queda com una taca blanca.



3. Llavors. Cada taca que té l'alçada d'un caràcter (entre el 50 % i el 90 % de la matrícula) i no toca la vora és una llavor. Si una taca fa més d'1,5 caràcters d'amplada, són dos caràcters enganxats, i hi posem una llavor per caràcter.



4. Watershed. Cada llavor s'estén pel seu caràcter i separa els que es toquen.
5. Comprovar que hi ha 7 caràcters.
    a. Si en surten més, traiem la regió que està menys alineada amb les altres (ajustem una recta a la part de dalt i una altra a la de baix dels caràcters). Així desapareix la peça del cotxe de la dreta de 0182GLK.



    b. Si en surten menys, és perquè la matrícula està retallada molt justa i els caràcters toquen la vora. En aquest cas provem l'obertura 3×3 en lloc del tancament 1×3, i si així en surten 7, ens la quedem.
6. Màscara binària i retall. La unió de les regions és la màscara dels caràcters, i crop_characters en retalla cada caràcter d'esquerra a dreta.



Resultats (com a referència, les caixes de l'Ex1, perquè no tenim la GT dels caràcters):

Llavors	Matrícules amb 7 caràcters (de 69)	Caràcters de l'Ex1 trobats un sol cop, train (frontal / lateral)
Màxims del LoG (Ex2c)	7	61,0 % / 64,7 %
Taques de mida de caràcter	42	100% / 96,4%
Mètode final	**69**	**100% / 100%**
Referència: Ex1 dels profes	68	(és la referència)

El mètode final segmenta 7 caràcters en les 19 matrícules frontals i en les 50 laterals (69 de 69), i no deixa cap regió fora d'un caràcter. La comprovació de «menys de 7» s'ha fet servir en 1 matrícula (1565HTS), on l'Ex1 dels profes també falla. L'Ex1 dels profes segmenta 68 de 69. A validation (2 frontals i 9 laterals) el mètode final també troba el 100 % dels caràcters de l'Ex1, mentre que amb les taques de mida de caràcter en troba el 93,7 % a la lateral.

Limitacions.

- Tot s'ha provat sobre els retalls del detector clàssic, redreçats a 400 px d'amplada. Al sistema final, l'entrada serà el retall de YOLO, que no està redreçat, i caldrà tornar-ho a comprovar.
- Les decisions s'han pres mirant matrícules de train.